

06-ansible 的条件判断

1 条件判断应用场景

在有的时候 play 的结果依赖于变量、fact 或者是前一个任务的执行结果，或者基于上一个 task 执行返回的结果而决定如何执行后续的 task，这个时候就需要用到条件判断。

条件语句在 Ansible 中的使用场景：

在目标主机上定义了一个硬限制，比如目标主机的最小内存必须达到多少，才能执行该 task

捕获一个命令的输出，根据命令输出结果的不同以触发不同的 task

根据不同目标主机的 facts，以定义不同的 task

根据目标机的 cpu 的大小，以调优相关应用性能

用于判断某个服务的配置文件是否发生变更，以确定是否需要重启服务

JSON 复制

when: 运算符

when 关键字后面的内容默认就会识别为变量，所以我们在引用变量的时候无论是自定义的、fact 事实变量、魔法变量都不要加上 "{{ }}"

2 比较运算符

2.1 数字比较运算符

Markdown 复制

>：比较两个对象的大小，左边的值大于右边的值，则为真
<：比较两个对象的大小，左边的值小于右边的值，则为真
>=：比较两个对象的大小，左边的值大于等于右边的值，则为真
<=：比较两个对象的大小，左边的值小于等于右边的值，则为真

当我们要安装软件包的时候，可以先提前判断被控节点剩下的磁盘空间是否足够

案例 1：当被控节点剩余空闲内存空间小于 500M 的时候，就不要执行 task 任务

YAML 复制

- name: when1
hosts: all
tasks:
- name: task1
debug:
msg: 这是我第一次使用when条件判断的语句

- name: task2
debug:
msg: ok
when: ansible_memfree_mb > 500

案例 2：当被控节点操作系统版本大于等于 8，那么就执行配置 2 个仓库的 task 任务

YAML 复制

[root@adsl-172-10-0-153 ansible]# cat when2.yaml
- name: when2
hosts: all
tasks:
- name: configure repository <= 7
yum_repository:
file: dvd
name: dvd
description: dvd
baseurl: file:///media
gpgcheck: 0

```

    enabled: 1
  when: ansible_distribution_major_version <= '7'
- name: configure repository  >= 8
  yum_repository:
    file: dvd
    name: BaseOS
    description: baseos
    baseurl: file:///media/BaseOS
    gpgcheck: 0
    enabled: 1
  when: ansible_distribution_major_version >= '8'

- name: configure repository  >= 8
  yum_repository:
    file: dvd
    name: AppStream
    description: appstream
    baseurl: file:///media/AppStream
    gpgcheck: 0
    enabled: 1
  when: ansible_distribution_major_version >= '8'
```

2.2 字符串比较运算符

Markdown 复制

==: 比较两个对象是否相等，相等则返回真。可用于比较字符串和数字

!=: 比较两个对象是否不等，不等则为真。

案例 1：当被控节点主机名是 node1 的时候，那么执行 debug 模块的任务，输出 node1 ok

YAML 复制

```
[root@adsl-172-10-0-153 ansible]# cat when3.yaml
- name: when3
  hosts: all
  tasks:
    - name: task1
      debug:
        msg: node1 ok
      when: inventory_hostname == 'node1'

    - name: task2
      debug:
        msg: 所有主机都会输出ok;
```

案例 2：当被控节点操作系统不是 RedHat 的时候，那么执行 debug 模块的任务，输出 not redhat

YAML 复制

```
- name: when3
  hosts: all
  tasks:
    - name: task1
      debug:
        msg: not redhat
      when: ansible_os_family != 'RedHat'
```

3 逻辑运算符

Markdown 复制

and: 逻辑与，当左边和右边两个表达式同时为真，则返回真

or: 逻辑或，当左右和右边两个表达式任意一个为真，则返回真

not: 逻辑否，对表达式取反

(): 当一组表达式组合在一起，形成一个更大的表达式，组合内的所有表达式都是逻辑与的关系

案例 1：只有当操作系统版本是 RedHat 并且主机的空闲内存大于 100M 的，才执行 task1 任务

YAML 复制

```
- name: when4
hosts: all
tasks:
- name: task1
  debug:
    msg: ok
  when: ansible_os_family == 'RedHat' and ansible_memfree_mb > 100
```

案例 2：主机名叫做 node1 的或者主机的内存空间大于 1000M 的，才执行 task1 任务

YAML 复制

```
- name: when4
hosts: all
tasks:
- name: task1
  debug:
    msg: ok
  when: inventory_hostname == 'node1' or ansible_memtotal_mb > 1000
```

案例 3：当主机操作系统是 RedHat 并且主机名是 node1 的，或者，主机剩余内存空间大于 100M 的并且操作系统版本是大于等 9 的，才会执行 task1 任务

YAML 复制

```
- name: when4
hosts: all
tasks:
- name: task1
  debug:
    msg: ok
  when: ( ansible_os_family == 'RedHat' and inventory_hostname == 'node1' ) or ( ansible_memfree_mb > 100 and ansible_distribution_major_version >= '9' )
```

4 配合 register 注册变量进行判断

register 注册变量和判断语句联合使用：都是用来判断前面一个 task 任务是否执行成功，然后再决定是否要执行后面的 task 任务

YAML 复制

```
- name: register
hosts: all
tasks:
- name: tasks1
  shell: rm -rf /tmp/*
  register: status

- name: use debug
  debug:
    msg: "{{ status }}"

- name: tasks2
  shell: touch /tmp/abc
  when: status.rc == 0
```

5 文件属性运算符

Markdown 复制

file：判断指定路径是否为一个文件，是则为真
directory：判断指定路径是否为一个目录，是则为真
link：判断指定路径是否为一个软链接，是则为真
mount：判断指定路径是否为一个挂载点，是则为真
exists：判断指定路径是否存在，存在则为真
特别注意：关于路径的所有判断均是判断主控端上的路径，而非被控端上的路径

案例：如果被控节点不存在/opt/passwd 文件，则把被控节点自己的/etc/passwd 拷贝过去

YAML 复制

```
- name: file
hosts: all
tasks:
- name: task
  shell: ls /opt/passwd
```

```
ignore_errors: yes
register: status

- name: task1
  copy:
    remote_src: yes
    src: /etc/passwd
    dest: /opt/passwd
    when: status.rc != 0
```

案例：如果控制节点不存在/opt/passwd 文件，则把被控节点自己的/etc/passwd 拷贝到/opt/目录下

YAML 复制

```
- name: file
  hosts: all
  tasks:
- name: task1
  copy:
    remote_src: yes
    src: /etc/passwd
    dest: /opt/passwd
  when: not '/opt/passwd' is exists
```

6 变量运算符

Markdown 复制

```
defined: 判断变量是否已定义，已定义则返回真
  os: version
undefined: 判断变量是否未定义，未定义则返回真

none: 判断变量的值是否为空，如果变量已定义且值为空，则返回真
  os:
```

如果你在使用别人开发的 playbook 的时候，没有传入参数，那么就不会执行任务的 task 任务，你就可以使用变量运算符来代替它

7 in 关键字判断

判断关键字是否在另外一个关键字中

判断内容是否在一个列表中

YAML 复制

```
- name: file
  hosts: all
  vars:
    name1:
      - a
      - b
      - c
  tasks:
- name: tasks
  debug:
    msg: ok
  when: "'a' in name1"
```

YAML 复制

如果被控节点ens160网卡存在，则配置网络信息

如果被控节点存在/dev/vdb设备，则进行分区

```
- name: file
  hosts: all
  vars:
    name1:
      - a
      - b
      - c
```

```
tasks:
- name: tasks
  debug:
    msg: ok
  when: "'nvme0n1' in ansible_devices"

- name: tasks
  debug:
    msg: ok
  when: ansible_devices.nvme0n1 is defined
```

8 根据任务执行结果判断

YAML 复制

```
- name: register
hosts: all
tasks:
- name: create user
  user:
    name: aabbcc
    state: present
  register: status

- name: use debug
  debug:
    msg: ok
  when: status is success  # 当任务执行成功则判断ok
```

配合register注册变量

```
status is success/changed/skip/failed
```

9 其他的运算符

YAML 复制

判断字符串：

lower：判断字符串中的所有字母是否都是小写，是则为真

upper：判断字符串中的所有字母是否都是大写，是则为真

示例：

```
- hosts: test
gather_facts: no
vars:
  str1: "abc"
  str2: "ABC"
tasks:
- debug:
  msg: "str1 is all lowercase"
  when: str1 is lower
- debug:
  msg: "str2 is all uppercase"
  when: str2 is upper
```

YAML 复制

判断整除：

even：判断数值是否为偶数，是则为真

odd：判断数值是否为奇数，是则为真

divisibleby(num)：判断是否可以整除指定的数值，是则为真

```
- hosts: test
gather_facts: no
vars:
  num1: 6
  num2: 8
  num3: 15
```

```
tasks:
  - debug:
      msg: "num1 is an even number"
    when: num1 is even
```

Markdown 复制

其他条件测试：

subset 判断一个list是不是另一个list的子集：when: a is subset(b)

superset 判断一个list是不是另一个list的父集：when: b is superset(a)

string 判断对象是否为一个字符串，是则为真 when: var1 is string

number 判断对象是否为一个数字，是则为真 when: var3 is number

10 block、rescue、always

when 判断中，可能多个任务使用的都是一个 when 判断语句来进行判断

比如；要执行 task1 任务，当主机名是 node1，要执行 task2 任务，当主机名是 node1

```
- name: task1
  debug:
    msg: task1
  when: inventory_hostname == 'node1'
- name: task2
  debug:
    msg: task2
  when: inventory_hostname == 'node1'
```

YAML 复制

既然多个 task 任务判断条件是一样的，我们可以借助 block 关键字把相同判断条件的任务包裹起来，然后 when 只针对于 block 做判断

```
- block:
  - name: task1
    debug:
      msg: task1

  - name: task2
    debug:
      msg: task2
  when: inventory_hostname == 'node1'
```

YAML 复制

block 配合 rescue 关键字实现错误的任务处理，就是说如果 block 中的任务有一个执行失败了，那么就会执行 rescue 关键字中的任务，如果 block 中没有任务执行失败，则不会执行 rescue 关键字中的任务

```
- name: block
  hosts: all
  tasks:
  - block:
    - name: task3
      debug:
        msg: taks3
    - name: task1
      shell: useradd root
  when: inventory_hostname == 'node1'
  rescue:
  - name: rescue1
    debug:
      msg: resuce1
```

YAML 复制

always 关键字，配合 block 使用的，无论 block 中的任务执行成功还是执行失败，最终都会执行 always 关键字中的任务

```
▼ [root@adsl-172-10-0-153 ansible]# cat block.yaml
[root@adsl-172-10-0-153 ansible]# cat block.yaml
- name: block
  hosts: all
```

YAML 复制

```
tasks:
- block:
  - name: task3
    debug:
      msg: taks3
  - name: task1
    debug:
      msg: taks1
rescue:
- name: rescue1
  debug:
    msg: resuce1
always:
- name: always
  debug:
    msg: always
when: inventory_hostname == 'node1'
```

block 实现的效果：

1. 可以将多个判断条件一样的任务包裹起来进行判断，这样的话就可以少写一些 when 判断了
2. block 配合 rescue 可以实现错误处理，block 任务执行失败了就会执行 rescue 的任务
3. block、rescue、always 三者一般联合使用